Nantes Université

Faculté des Sciences et Techniques de Nantes
Licence d'informatique – 2ème année

Rapport de conception du projet de programmation objet

Titre de votre projet : casino magnifito

22 novembre 2024

Équipe POO.24.25.9 :

- Prénom Nom Djeutcheu nono lise benila
- Prénom Nom Estaben Mathieu
- Prénom Nom Vigneau Juliette

Encadrant du TP XXX :

- Laurent Granvilliers

1 Cahier des charges

Contexte : Ce projet met en œuvre un casino immatériel, dont les machines qu'on peut débloquer en jouant deviennent de plus en plus risquées.

Objectifs : Le joueur pourra, en partant d'une mise de départ, jouer pour essayer de faire augmenter son nombre de jetons en misant.

Fonctions applicatives : Notre algorithme offrira plusieurs possibilités, telles que plusieurs niveaux de difficulté pour les machines. Ces dernières seront déblocables par un temps de jeu, des missions, un niveau de joueur (système de progression associé au joueur). Ce même joueur pourra jouer une mise (l'augmenter ou la baisser), cette même mise définira le gain d'expérience qui influera sur son niveau.

Cas d'utilisation : Au début d'une partie, le joueur part d'une mise donnée, et peut jouer à une machine. A mesure qu'il fait des parties, il gagne des points d'expérience qui lui permettent de débloquer de nouvelles fonctionnalités qui facilitent le gameplay. Il gagne aussi naturellement de l'ancienneté à mesure que le temps passe, ce qui lui permet de débloquer des machines plus risquées (grand risque, grandes récompenses).

2 Architecture

2.1 Description générale

Nous apporterons java.util.random pour gérer l'aléa.

Interface Machine :

Méthodes :
- lancer() ; Change les rouleaux/le résultat du lancer.
- afficherResultat() ; Affiche le résultat de lancer()
- vérifierResultat() ; Vérifie les alignements, le rapport pari/resultat pour calculer un gain ou une perte.
- gains() ; (Augmente la cagnotte du joueur, augmente le "resultat" sur la machine du nombre gagné, affiche la gain après l'appel à Lancer() ;)

Classe Joueur :

Une chaîne de caractère pour le nom du joueur qu'il choisira au début. Ce nom sera affiché dans un coin de l'interface que nous appellerons "profil". Un entier pour le niveau du joueur, un autre entier qui représentera le prochain passage à niveau, cet entier augmentera de 50pourcent à chaque passage à niveau. Un autre entier représentera la "barre" d'expérience et augmentera à chaque appel de la méthode "Lancer()" jusqu'à atteindre l'entier précédent et augmenter le niveau du joueur. A la manière du nom, le niveau sera également dans la partie "profil".

Méthodes :
- nommer() ; Permet de prendre un nom au lancement du programme.
- getSolde() ; Renvoie la valeur du solde du joueur.
- ajouterSolde() ; Vérifie que le montant à ajouter (ou retirer) au solde est positif puis ajoute.

— retirerSolde() ; Vérifie que le montant à retirer (ou ajouter) au solde est négatif puis retire.
— ajouterExp() ; Ajoute de l'expérience au joueur.
— afficherInfo() ; Affiche les infos précédentes.

De plus : Stockage des gains de chaque machine par rapport au nombre de lancer afin d'afficher un graphique lorsqu'on appuie sur un bouton "Stats" afin de comparer les performances sur les 3 machines.

Class Roulette :

1. attributs ;

— état (si la machine est allumée ou pas)

— Deux listes de nombre pour les deux couleurs différentes. une liste redNumbers et une blackNumbers.

— Un niveau d'expérience nécessaire pour jouer à cette machine.

Le but de la classe sera juste de gérer les numéros de la roulette, effectuer le tirage (avec la rotation de la roulette en utilisant la fonction random) et déterminer le numéro ou la couleur gagnante. le joueur pourra parier sur un numéros ou sur une couleur.

1. méthodes ;

— lancer le jeux

— afficherResultat

— gains

— peutJouer ; vérifie si le joueur a le niveau d'expérience requis pour jouer sur cette machine

— definirMise : enregistre la mise du joueur avant de jouer. la mise est en fonction des jetons du joueur.

— effectuerPari ; le joueur effectue un pari soit sur un nombre, soit sur une couleur.À l'aide de la fonction random, on simule un tour, on prend un numéro aléatoire et on compare le pari du joueur avec le résultat de la simulation. De plus, on met à jour le solde du joueur (on ajoute les jetons du joueur en fonction du gain).

— jouer : effectue une partie complete. avec la mise, affiche le resultat et les gains

les méthodes lancer le jeux, afficherResultat et gains sont hérité de l'interface machine.

Classe All in : (possibilité de miser tous les jetons du joueur)

Un tableau de trois cases, case toute à gauche est parmis (+, -, *, /), la case au milieu et toute à droite sont des chiffres. Si le symbole obtenu est un + ou un -, les deux chiffres obtenus sont multipliés puis ce nombre est additionné ou soustrait à la mise. Si le symbole obtenu est un * ou un /, alors le chiffre le plus petit des deux obtenus est alors multiplié par la mise ou divise la mise.

1. attributs :

— nom de la machine,
— tableau de trois cases,
— le niveau de difficulté,
— le gain,
— la mise,

— le nombre de victoires,
— le nombre de défaites,

1. méthodes :

— lancer() : public, reset le tableau puis détermine niveau de difficulté, puis (avec appel méthode trouverSymbole et fonction random de la bibliothèque util) trouve les deux chiffres et le symbole
— afficherResultat() : public, affichage du tableau avec les chiffres et le symbole proprement
— verifierResultat() : public, calcul du gain (+, - , * ou /) en fonction des deux chiffres et du symbole : + ou - avec multiplication des deux chiffres puis + ou - sur la mise, et * ou / avec choix du plus petit chiffre des deux et * ou / sur la mise.
— gains() : public, affichage de la mise restante à la fin d'un tour proprement
— trouverSymbole1() : private, niveau difficulté facile, trouve un chiffre entre 0 et 9 au hasard, de 0 à 5 = +, de 6 à 9 = -
— trouverSymbole2() : private, niveau difficulté moyenne, trouve un chiffre entre 0 et 9 au hasard, de 1 à 3 = +, de 4 à 9 = -, si 0 = *
— trouverSymbole3() : private, niveau difficulté élevé, trouve un chiffre entre 0 et 9 au hasard, de 0 à 4 = /, de 5 à 9 = *
— mise() : public, (mise par défaut pour chaque machine), le joueur peut ensuite décider de miser plus ou de miser moins à chaque nouveau tour, le gain est calculé à partir de la mise à chaque tour
— niveauDifficulté() : public, quand lancement d'une nouvelle partie, le joueur a le choix entre trois niveaux de difficulté (question, prise en compte du choix, retourne 1, 2 ou 3, qui va déterminer quel algo trouverSymbole va être utilisé)
— nombreAleatoire() : private, quand appelé, retourne un nombre aléatoire entre 0 et 9 en utilisant la fonction random(), venant de la bibliothèque util de Java

class Magic Moolah Extravaganza (Machine 4x3) :

1. Attributs :

— Un tableau de tableau, le premier de taille 3, le second de taille 4 qui représente les résultats de la machine.
— La mise représente le prix pour lancer une partie qui décide également combien le joueur peut gagner.
— Le gain représente le nombre de jetons que gagne le joueur à la fin de sa partie (Il n'y en a pas si aucune combinaison n'est pas reconnue).
— Le gain total représente la totalité des gains du joueur moins toutes ses mises.
— Une description du fonctionnement de la machine.
— Un indice de volatilité qui représente le niveau de risque de la machine. A afficher à la sélection de la machine.
— Le gain maximal possible de la machine par rapport à la mise. A afficher à la sélection de la machine.

1. Méthodes :

— lancer() ; Public, reset le tableau et le rempli de chiffres et de symboles aléatoires. Appel la méthode ajouterExp() dans Joueur.
— multiples méthodes "événements" qui seront privées (Si le tableau contient une suite identique de symboles sur une même ligne, on va faire appel à la méthode qui lui correspond).

— modifMise() ; Public (Augmente ou baisse la mise selon si le joueur a appuyé sur le + ou le - du bouton mise permettant ainsi au joueur de jouer plus ou moins sur la machine à chaque lancer).
— getGains() Public, retourne les gains après chaque partie.
— getMise() Public, retourne la mise actuelle.
— getGainsTotaux() Public, retourne les gains de la machine depuis le lancement du programme.

Classe Casino :

Attributs :
— liste de machine de taille 3.
Methodes :
— AjouterMachine(Machine m) ; Public, permet d'ajouter une machine à la liste des machines afin de donner l'accès au joueur aux machines. Cette méthode sera appelée lorsque la condition pour accéder à une machine (Au niveau 2 par exemple) sera atteinte.
— AfficherMachine() ; Public, retourne la liste des machines.

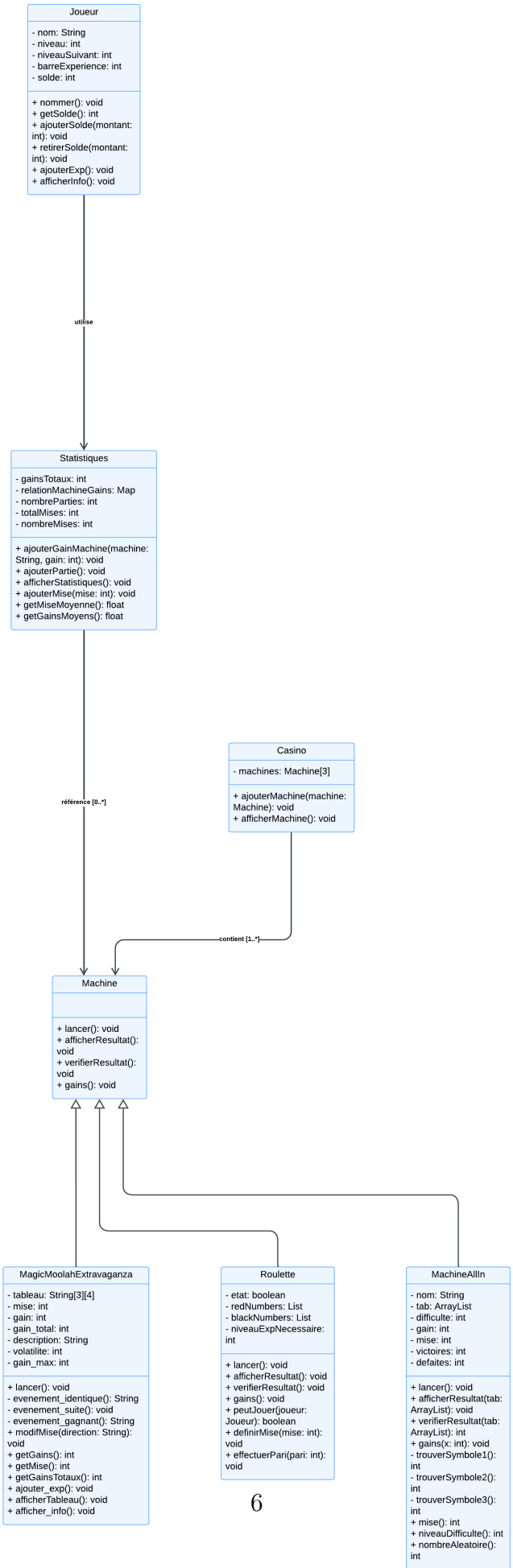
Classe Statistiques :

Attributs :
— Un entier qui représente les gains totaux du joueur.
— Une relation binaire qui représente le nom d'une machine et les gains du joueur sur cette machine.
— Un entier pour le nombre de parties.
— Un entier pour la valeur totale des mises.
— Un entier pour le nombre de mises.
Méthodes :
— ajouterGainMachine() ; Ajoute le gain d'une partie aux statistiques associées.
— ajouterPartie() ; Ajoute une partie au nombre total de partie.
— afficherStatistiques() ; Affiche les statistiques.
— ajouterMise() ; Ajoute la mise au nombre total de mise et à la valeure total des mises.
— getMiseMoyenne() ; Retourne la moyenne des mises.
— getGainsMoyen() ; Retourne les gains moyens.

2.2 Diagramme de classes

2.3 Interfaces

L'interface du casino Magnifico sera conçue pour être interactive, le joueur pourra en effet naviguer entre les machines. Cette interface sera composée : D'un écran d'accueil où sera affiché le profil de joueur avec son nom, son solde, son niveau et ses statistiques de jeu. Un écran de sélection de machine où seront disposées les machines débloquées avec leur gain maximum affiché ainsi que leur indice de volatilité. Un écran pour chaque machine qui contiendra les rouleaux, un bouton pour lancer une partie, un bouton + et - pour la mise. Les gains seront affichés à la fin d'une partie. Un tableau de bord qui contient les statistiques de toutes les machines sera accessible n'importe quand avec un bouton.


Joueur
- nom: String
- niveau: int
- niveauSuivant: int
- barreExperience: int
- solde: int
+ nommer(): void
+ getSolde(): int
+ ajouterSolde(montant: int): void
+ retirerSolde(montant: int): void
+ ajouterExp(): void
+ afficherInfo(): void
utilise
Statistiques
- gainsTotaux: int
- relationMachineGains: Map
- nombreParties: int
- totalMises: int
- nombreMises: int
+ ajouterGainMachine(machine: String, gain: int): void
+ ajouterPartie(): void
+ afficherStatistiques(): void
+ ajouterMise(mise: int): void
+ getMiseMoyenne(): float
+ getGainsMoyens(): float
référence [0..*]
Casino
- machines: Machine[3]
+ ajouterMachine(machine: Machine): void
+ afficherMachine(): void
contient [1..*]
Machine
+ lancer(): void
+ afficherResultat(): void
+ verifierResultat(): void
+ gains(): void
MagicMoolahExtravaganza
- tableau: String[3][4]
- mise: int
- gain: int
- gain_total: int
- description: String
- volatilite: int
- gain_max: int
+ lancer(): void
- evenement_identique(): String
- evenement_suite(): void
- evenement_gagnant(): String
+ modifMise(direction: String): void
+ getGains(): int
+ getMise(): int
+ getGainsTotaux(): int
+ ajouter_exp(): void
+ afficherTableau(): void
+ afficher_info(): void
Roulette
- etat: boolean
- redNumbers: List
- blackNumbers: List
- niveauExpNecessaire: int
+ lancer(): void
+ afficherResultat(): void
+ verifierResultat(): void
+ gains(): void
+ peutJouer(joueur: Joueur): boolean
+ definirMise(mise: int): void
+ effectuerPari(pari: int): void
MachineAllIn
- nom: String
- tab: ArrayList
- difficulte: int
- gain: int
- mise: int
- victoires: int
- defaites: int
+ lancer(): void
+ afficherResultat(tab: ArrayList): void
+ verifierResultat(tab: ArrayList): int
+ gains(x: int): void
- trouverSymbole1(): int
- trouverSymbole2(): int
- trouverSymbole3(): int
+ mise(): int
+ niveauDifficulte(): int
+ nombreAleatoire(): int

2.4 Aspects spécifiques

Il y a selon nous plusieurs aspects spécifique à notre projet :

1. La génération de symboles et de nombre aléatoire pour remplir les rouleaux, cette génération inclut donc une gestion particulière des probabilités, par exemple l'apparition des symboles '+', '-', '*' et '/' dans la machine all-in.
2. Le calcul des gains est spécifique à chaque machine et à leurs règles spécifiques.
3. La gestion de la progression du joueur, que ce soit avec l'augmentation en niveau à chaque partie ou avec le temps de jeu qui augmente à partir du moment où le joueur lance le programme.

3 Regard critique

— Mise en oeuvre des concepts de la poo :

Notre projet adopte les principes de la programmation orientée objet avec des classes, chacune de ces classes encapsule ses données et ses méthodes. Nous utilisons également l'héritage avec l'interface Machine. Calibrage du projet et la quantité du travail fournie : Notre projet semble bien calibré, nos machines ont une répartition plutôt logique des tâches. La quantité de travail est importante en raison de la gestion de plusieurs machines, mais ça tombe bien, nous formons un groupe, nous nous sommes assigné chacun une machine ce qui nous permettra d'être efficace.

— Travail, motivation et organisation de l'équipe :

Nous pensons nous être organisés correctement, comme expliqué précédemment en ayant chacun une machine nous pouvons progresser à la même vitesse. Au niveau motivation, elle reste élevée car le sujet du projet nous intéresse et notre progression dans celui-ci étaye notre créativité, en effet cela risque même d'être un problème si nous venons à ajouter trop de fonctionnalités sans pouvoir en assumer les conséquences par la suite.

— Compétences de l'équipe au début du projet et développement de nouvelles compétences :

Après avoir écris le rapport du projet et réfléchis à sa conception, nous savons maintenant que l'équipe va s'améliorer et développer plusieurs points :

— La création d'interfaces graphiques
— La conception d'un système de progression
— L'utilisation des probabilités en informatique

En effet, avant le projet, aucun membre du groupe n'a créé d'interface graphique par le passé en java. De plus, l'implémentation et la génération de nombre aléatoires ont été très rares dans notre parcours informatique.

— Perspectives d'extension du projet et extensibilité de la solution actuelle :

Le projet promet une excellente extensibilité, en effet nous pouvons ajouter autant de machines que nous le voudront, pareil pour des fonctionnalités que ce soit pour la progression du joueur ou directement dans les machines. L'interface machine simplifie une possible extension. L'interface graphique peut être améliorée, que ce soit au niveau de la créativité des fenêtres machines, l'animation des parties. Pour conclure, nous pensons que ce projet a une extensibilité exceptionnelle en plus d'être captivant.